

SNAPFIX

Developer Reference Document

5 Releases · Personal Project

Release	Focus
Release 1	Civic Reporting Foundation
Release 2	Worker Marketplace & Task Assignment
Release 3	Task Completion, Verification & Payment
Release 4	AI Intelligence & Event-Driven Architecture
Release 5	Production Hardening & Scalability

Release 1 — Civic Reporting Foundation

Goal: Citizens can register, report infrastructure issues with images + geolocation, and nearby reports are discoverable. Workers can see reports near them.

Tech Stack

Layer	Technology
Backend	Spring Boot, Spring Security, Spring Data JPA
Database	PostgreSQL + PostGIS extension
Storage	Cloudinary (image upload + URL storage)
Frontend	React
DevOps	Docker, GitHub Actions
Testing	JUnit, Mockito, TestContainers, REST Assured

Module Structure

```
snapfix-backend/  
├── auth-module  
├── user-module  
├── report-module  
├── notification-module  
├── storage-module  
└── geo-module
```

⚠️ **Why modular monolith?** Faster to build and debug. Individual modules can be extracted into microservices later.

Phase 1 — Platform Foundation

Set up the base project infrastructure. Nothing functional yet — just the skeleton everything else builds on.

Base Entities

BaseEntity (abstract)		
Field	Type	Notes
id	UUID	Auto-generated primary key
createdAt	Instant	Auto-set on persist
updatedAt	Instant	Auto-set on update

User		
Field	Type	Notes
id	UUID	Extends BaseEntity

email	String	Unique, not null
passwordHash	String	BCrypt hashed
role	Role (enum)	CITIZEN / WORKER / ADMIN

Location		
Field	Type	Notes
point	Point	PostGIS geometry type — stores lat/lng

Non-Functional Targets (Phase 1 Baseline)

Metric	Target
Reports	1,000 reports supported
Concurrent users	100
API response time	< 200ms

Iteration 1 — Project Bootstrapping

Deliverables: Running Spring Boot service, Postgres connected, health check at `GET /actuator/health`

Tasks

1. Create Spring Boot project with modules as separate packages
2. Add `docker-compose.yml` with Postgres + backend containers
3. Configure `application.yml` — datasource, JPA, Hibernate DDL
4. Add PostGIS extension: `CREATE EXTENSION IF NOT EXISTS postgis;`
5. Verify `GET /actuator/health` returns 200

Iteration 2 — Core Infrastructure

Deliverables: Cloudinary image upload working, structured JSON logs, global error responses

Tasks

6. Add Cloudinary SDK dependency, configure API key/secret via env vars
7. Create `StorageService` with `uploadImage(MultipartFile) → String url`
8. Create `GlobalExceptionHandler` with `@RestControllerAdvice`
9. Add structured logging with MDC for request tracing (use SLF4J + Logback)

Phase 1 — Definition of Done

Before moving to Phase 2, verify:
☐ Docker Compose starts cleanly (<code>docker-compose up</code>)
☐ Postgres connection established — no startup errors
☐ Image uploads to Cloudinary — URL returned

☐ Any invalid request returns structured JSON error

☐ Test coverage > 60%

Phase 2 — Identity & User System

Enable registration, login, JWT auth, and role-based access. Three actor types: **CITIZEN**, **WORKER**, **ADMIN**.

Entities

CitizenProfile		
Field	Type	Notes
userId	UUID	FK to User
name	String	
location	Location	Home/default location
reportsSubmitted	int	Counter

WorkerProfile (basic)		
Field	Type	Notes
userId	UUID	FK to User
name	String	
skills	List<String>	Stored as JSON array
rating	Double	Aggregate — updated in Release 3

API Endpoints

Method	Endpoint	Description
POST	/auth/register	Register new user. Returns JWT.
POST	/auth/login	Login. Returns access + refresh tokens.
POST	/auth/refresh	Exchange refresh token for new access token.
GET	/user/me	Get current user profile. Requires JWT.
PUT	/user/profile	Update profile fields.

JWT Implementation Notes

- Use `io.jsonwebtoken:jjwt` library
- Access token TTL: **15 minutes**. Refresh token TTL: **7 days**
- Store refresh tokens in DB — invalidate on logout
- Add `JwtAuthFilter` extends `OncePerRequestFilter` to Spring Security filter chain
- Annotate endpoints with `@PreAuthorize("hasRole('CITIZEN')")` etc.

Phase 2 — Definition of Done

Before moving to Phase 3, verify:

☐ POST /auth/register creates user in DB with hashed password
☐ POST /auth/login returns valid JWT
☐ GET /user/me returns 401 without token
☐ ADMIN endpoint returns 403 for CITIZEN role
☐ Refresh token flow works end-to-end

Phase 3 — Civic Report System

Core product feature. Citizens submit reports with image, description, category, and GPS. Reports are discoverable by nearby users.

Entity

Report		
Field	Type	Notes
id	UUID	
citizenId	UUID	FK to User
imageUrl	String	Cloudinary URL
description	String	Max 1000 chars
category	Category (enum)	POTHOLE, STREETLIGHT, GARBAGE, WATER_LEAK, ROAD_DAMAGE
location	Point	PostGIS geometry — lat/lng
status	ReportStatus (enum)	CREATED → IN_PROGRESS → COMPLETED
supportCount	int	Incremented when others report same issue
createdAt	Instant	

API Endpoints

Method	Endpoint	Description
POST	/reports	Create report. Accepts multipart/form-data (image + JSON fields).
GET	/reports/nearby	Get reports within radius. Query params: lat, lng, radius (metres, default 5000).
GET	/reports/{id}	Get single report by ID.
POST	/reports/{id}/support	Increment support count. One support per user per report.

PostGIS Geo Query

```
-- Find reports within radius (metres)
SELECT * FROM reports
WHERE ST_DWithin(
  location::geography,
```

```

    ST_MakePoint(:lng, :lat)::geography,
    :radiusMetres
)
ORDER BY ST_Distance(location::geography, ST_MakePoint(:lng, :lat)::geography);

```

⚠ Use `::geography` cast for accurate metre-based distance. Without it, `ST_DWithin` uses degrees.

Duplicate / Support Logic

- On `POST /reports`: before saving, run geo query within **50 metres**, same category
- If match found → call `POST /reports/{existingId}/support` instead of creating new report
- Return `HTTP 200` with existing report + message "Existing report found – your support has been added"

Phase 3 — Definition of Done

Before moving to Phase 4, verify:

- ☐ `POST /reports` creates report, uploads image, stores PostGIS point
- ☐ `GET /reports/nearby` returns correct results with varying radius
- ☐ Support count increments correctly — duplicate prevention works
- ☐ Geo query runs in < 200ms against 1,000 seeded reports
- ☐ Test coverage > 65%

Phase 4 — Discovery & Basic Notifications

Workers can discover nearby reports. Basic in-app notifications are stored when key events happen.

Entity

Notification		
Field	Type	Notes
<code>id</code>	<code>UUID</code>	
<code>recipientId</code>	<code>UUID</code>	FK to User
<code>type</code>	<code>String</code>	e.g. <code>REPORT_CREATED</code> , <code>BID_APPROVED</code>
<code>message</code>	<code>String</code>	
<code>read</code>	<code>boolean</code>	Default false
<code>createdAt</code>	<code>Instant</code>	

API Endpoints

Method	Endpoint	Description
<code>GET</code>	<code>/workers/reports/nearby</code>	Reports within 5km of worker current location.
<code>GET</code>	<code>/notifications</code>	Get notifications for current user. Supports <code>?unread=true</code> .

PATCH	<code>/notifications/{id}/read</code>	Mark notification as read.
--------------	---------------------------------------	----------------------------

Notification Trigger Points

Event	Recipients Notified
Report created	Workers within 5 km
Report supported	Original report owner (citizen)

⚠ Notifications in this phase are fire-and-forget synchronous calls inside service methods. They will be replaced by events in Release 4.

Release 1 — Final Checklist

Release 1 complete when:
☐ Auth flow works: register → login → JWT → protected endpoint
☐ Report creation: image → Cloudinary, GPS → PostGIS, fields → DB
☐ Nearby search: reports returned sorted by distance
☐ Support: count increments, duplicate detection in place
☐ Notifications: stored on report creation, retrievable
☐ All containers start cleanly with docker-compose up
☐ Test coverage > 70%

Release 2 — Worker Marketplace & Task Assignment

Goal: Workers bid on reports. Admin approves one bid. System auto-creates a Task. Workflow: Report → Bid → Task → IN_PROGRESS.

New Modules

```
snapfix-backend/
├── ... (existing)
├── worker-module
├── bid-module
├── task-module
└── admin-module
```

Phase 1 — Worker System

Expand the worker profile and add location tracking. Workers can now browse reports near them.

Entity

WorkerProfile (expanded)		
Field	Type	Notes
userId	UUID	FK to User
skills	List<String>	
rating	Double	Updated by rating service in R3
completedTasks	int	
walletBalance	BigDecimal	Used in Release 3
currentLocation	Point	PostGIS — updated on each location ping
available	boolean	Worker can toggle this

API Endpoints

Method	Endpoint	Description
POST	/workers/profile	Create worker profile (after registration).
PUT	/workers/profile	Update skills, availability.
GET	/workers/profile	Get own profile.
POST	/workers/location	Update current GPS location. Body: { lat, lng }.
GET	/workers/reports/nearby	Reports within 5km. Auto-expands to 10km if < 5 results.

⚠ **Early-stage rule:** Workers can bid on any report regardless of skill match. Skill-based filtering will be added later.

Phase 1 — Definition of Done

☐ Worker profile CRUD working
☐ Location stored as PostGIS Point
☐ Nearby reports query works with auto-expand logic

Phase 2 — Bidding Marketplace

Entity

Bid		
Field	Type	Notes
id	UUID	
reportId	UUID	FK to Report
workerId	UUID	FK to User
bidAmount	BigDecimal	Proposed cost
durationEstimate	int	Estimated hours
resourceNote	String	Optional — materials needed
status	BidStatus (enum)	ACTIVE / WITHDRAWN / APPROVED / REJECTED
createdAt	Instant	

Bid State Machine

From	To	Trigger
ACTIVE	WITHDRAWN	Worker calls DELETE /bids/{id} before approval
ACTIVE	APPROVED	Admin approves this bid
ACTIVE	REJECTED	Admin approves a different bid on same report

API Endpoints

Method	Endpoint	Description
POST	/bids	Place bid. Body: { reportId, bidAmount, durationEstimate, resourceNote }.
DELETE	/bids/{bidId}	Withdraw bid. Only allowed while status = ACTIVE.
GET	/bids/my	Worker views their own bids.
GET	/reports/{id}/bids	Admin views all bids for a report.

Validation Rules

- Report must exist and have status `CREATED`
- Worker must have a WorkerProfile
- `UNIQUE(reportId, workerId)` constraint — one bid per worker per report
- Withdrawal only allowed when `status == ACTIVE`

Phase 2 — Definition of Done

☐ Worker can place bid — stored in DB
☐ Duplicate bid returns 409 Conflict
☐ Worker can withdraw bid (status → WITHDRAWN)
☐ Admin can list all bids for a report

Phase 3 — Admin Governance

API Endpoints

Method	Endpoint	Description
GET	/admin/reports	List all reports (paginated). Filter by status.
GET	/admin/reports/{id}/bids	View all bids for a report.
POST	/admin/bids/{bidId}/approve	Approve bid → auto-rejects all others on same report.
POST	/admin/bids/{bidId}/reject	Manually reject a specific bid.

Approval Transaction (atomic)

10. Set chosen bid status → `APPROVED`
11. Set all other bids on same report → `REJECTED`
12. Trigger task creation (Phase 4)
13. Write to `AdminActionLog`

⚠ Wrap steps 1–4 in a single `@Transactional` method. If task creation fails, the whole operation rolls back.

Entity

AdminActionLog		
Field	Type	Notes
id	UUID	
adminId	UUID	FK to User
action	String	e.g. BID_APPROVED, TASK_REASSIGNED
targetId	UUID	ID of affected entity
note	String	Optional reason/comment
timestamp	Instant	

Phase 3 — Definition of Done

☐ Only ADMIN role can access /admin/** endpoints
☐ Approving one bid rejects all others atomically
☐ AdminActionLog entry created for every admin action

Phase 4 — Task Lifecycle Engine

Entity

Task		
Field	Type	Notes
id	UUID	
reportId	UUID	FK to Report — one task per report
workerId	UUID	FK to User (worker)
assignedAt	Instant	Set on creation
status	TaskStatus (enum)	See state machine below
retryCount	int	Default 0. Max 3 (enforced in R3)

Task State Machine (all releases)

From	To	Trigger
(none)	ASSIGNED	Admin approves bid
ASSIGNED	IN_PROGRESS	Worker calls PATCH /tasks/{id}/start
IN_PROGRESS	PROOF_SUBMITTED	Worker uploads proof (Release 3)
PROOF_SUBMITTED	VERIFIED_BY_CITIZEN	Citizen verifies repair (Release 3)
PROOF_SUBMITTED	REJECTED	Citizen rejects repair (Release 3)
REJECTED	IN_PROGRESS	Worker retries (Release 3, max 3 attempts)
VERIFIED_BY_CITIZEN	COMPLETED	Admin final approval (Release 3)
COMPLETED	PAYMENT_RELEASED	Admin releases payment (Release 3)

API Endpoints

Method	Endpoint	Description
GET	/workers/tasks	Worker views their assigned tasks.
GET	/tasks/{id}	Get task detail.
PATCH	/tasks/{id}/start	Worker marks task IN_PROGRESS.

Task Auto-Creation Logic

```
// Inside BidService.approveBid(bidId) — runs in @Transactional
bid.setStatus(BidStatus.APPROVED);
```

```
rejectAllOtherBids (bid.getReportId(), bidId);

Task task = new Task();
task.setReportId(bid.getReportId());
task.setWorkerId(bid.getWorkerId());
task.setStatus (TaskStatus.ASSIGNED);
taskRepository.save (task);

report.setStatus (ReportStatus.IN_PROGRESS);
reportRepository.save (report);
```

Release 2 — Final Checklist

Release 2 complete when:
☐ Worker profile + location tracking working
☐ Worker can place and withdraw bids
☐ Admin can view and approve bids
☐ Bid approval atomically rejects others and creates task
☐ Worker can view task and mark it IN_PROGRESS
☐ AdminActionLog records all admin decisions
☐ Test coverage > 75%

Release 3 — Task Completion, Verification & Payment

Goal: Full task resolution lifecycle. Worker submits proof → citizen verifies → admin approves → payment released to worker wallet.

New Modules

```
snapfix-backend/  
├── ... (existing)  
├── proof-module  
├── verification-module  
├── payment-module  
├── wallet-module  
└── rating-module
```

Phase 1 — Proof of Work System

Entity

Proof		
Field	Type	Notes
id	UUID	
taskId	UUID	FK to Task — one proof per task
workerId	UUID	Must match task.workerId
imageUrl	String	Cloudinary URL
gpsLocation	Point	GPS at time of submission
remarks	String	Optional worker notes
submittedAt	Instant	

API Endpoints

Method	Endpoint	Description
POST	/tasks/{taskId}/proof	Upload proof. Multipart: image + location + remarks. Task must be IN_PROGRESS.
GET	/tasks/{taskId}/proof	View proof. Accessible by task worker, citizen, and admin.

Validation

- `task.workerId` must match authenticated user
- `task.status` must be `IN_PROGRESS`
- Image file required — reject if missing
- On success: set `task.status = PROOF_SUBMITTED`

Phase 1 — Definition of Done

☐ Worker submits proof → image in Cloudinary, metadata in DB
☐ Task status moves to PROOF_SUBMITTED
☐ Wrong worker gets 403
☐ Task not IN_PROGRESS gets 409 Conflict

Phase 2 — Citizen Verification Workflow

Entity

Verification		
Field	Type	Notes
id	UUID	
taskId	UUID	FK to Task
citizenId	UUID	Must be original report submitter
status	VerificationStatus (enum)	VERIFIED / REJECTED
comments	String	Optional reason for rejection
timestamp	Instant	

API Endpoints

Method	Endpoint	Description
POST	/tasks/{taskId}/verify	Body: { status: "VERIFIED" "REJECTED", comments }. Citizen only.

State Transitions

From	To	Trigger
PROOF_SUBMITTED	VERIFIED_BY_CITIZEN	Citizen submits VERIFIED
PROOF_SUBMITTED	REJECTED	Citizen submits REJECTED

Auto-Verification (Inactivity)

- Schedule a job using @Scheduled (Spring) to run daily
- Find tasks in PROOF_SUBMITTED state for > 5 days
- Auto-set status → VERIFIED_BY_CITIZEN
- Store a system-generated Verification record with note: "Auto-verified due to citizen inactivity"

Phase 2 — Definition of Done

--

☐ Citizen can verify or reject a proof
☐ Non-citizen of that report gets 403
☐ Task status transitions correctly on both paths
☐ Scheduler auto-verifies after 5 days of inactivity

Phase 3 — Retry & Exception Handling

Retry Rules

Rule	Value
Max retries	3
Field tracked	<code>task.retryCount</code>
Allowed when	Task status = REJECTED
After limit exceeded	Task locked, admin must intervene

API Endpoints

Method	Endpoint	Description
POST	<code>/tasks/{taskId}/retry</code>	Worker retries repair. Increments retryCount. Sets status → IN_PROGRESS.

Retry Logic

```
// Inside TaskService.retry(taskId)
if (task.getStatus() != TaskStatus.REJECTED) throw new InvalidStateException();
if (task.getRetryCount() >= 3) throw new RetryLimitExceededException();

task.setRetryCount(task.getRetryCount() + 1);
task.setStatus(TaskStatus.IN_PROGRESS);
taskRepository.save(task);
```

Phase 3 — Definition of Done

☐ Worker can retry up to 3 times after rejection
☐ 4th retry attempt returns 409
☐ retryCount persists correctly

Phase 4 — Admin Final Approval

API Endpoints

Method	Endpoint	Description
GET	/admin/tasks	List tasks. Filter by status.
GET	/admin/tasks/{id}	Task detail with proof + verification.
POST	/admin/tasks/{taskId}/approve	Final approval. Status → COMPLETED.
POST	/admin/tasks/{taskId}/reject	Reject completion. Status → REJECTED.
POST	/admin/tasks/{taskId}/reassign	Body: { newWorkerId }. Creates new task assignment.

⚠ Admin approval required before payment is released. POST /admin/tasks/{id}/approve triggers the payment pipeline.

Phase 5 — Payment & Wallet System

Entities

Wallet		
Field	Type	Notes
id	UUID	
workerId	UUID	FK to User — one wallet per worker
balance	BigDecimal	Always >= 0

Transaction		
Field	Type	Notes
id	UUID	
walletId	UUID	FK to Wallet
amount	BigDecimal	
type	String	CREDIT / DEBIT
referenceId	UUID	FK to Payment or other source
timestamp	Instant	

Payment		
Field	Type	Notes
id	UUID	
taskId	UUID	FK to Task
workerId	UUID	
amount	BigDecimal	Taken from approved bid.bidAmount
status	PaymentStatus (enum)	PENDING / RELEASED / FAILED
releasedAt	Instant	

API Endpoints

Method	Endpoint	Description
GET	/workers/wallet	Get own wallet balance.
GET	/workers/payments	Payment history for current worker.
POST	/admin/payments/{taskId}/release	Release payment for completed task. Admin only.

Payment Release Flow (atomic)

14. Find `Payment` record for task (created when task was set COMPLETED)
15. Set `payment.status = RELEASED`
16. Add `bid.bidAmount` to `wallet.balance`
17. Create `Transaction` record (type: CREDIT)
18. Send notification to worker

⚠ Use `@Transactional` on the payment release method. Payment amount = `bid.bidAmount` from the approved bid.

Phase 5 — Definition of Done

☐ Wallet auto-created when worker profile is created
☐ Admin releases payment → balance updated atomically
☐ Transaction record created for every payment
☐ Worker sees updated balance and payment history

Phase 6 — Worker Reputation System

Entity

Rating		
Field	Type	Notes
<code>id</code>	UUID	
<code>taskId</code>	UUID	FK to Task — one rating per task
<code>workerId</code>	UUID	
<code>citizenId</code>	UUID	Must be original report submitter
<code>score</code>	int	1–5 stars
<code>comment</code>	String	Optional
<code>timestamp</code>	Instant	

API Endpoints

Method	Endpoint	Description
--------	----------	-------------

POST	/workers/{workerId}/rating	Submit rating. Only allowed if task is COMPLETED and user is original citizen. Body: { taskId, score, comment }.
GET	/workers/{workerId}/rating	Get worker rating summary: average score, task count.

Rating Aggregation

```
// Update WorkerProfile after new rating saved
double avg = ratingRepository.findAverageScoreByWorkerId(workerId);
worker.setRating(avg);
worker.setCompletedTasks(worker.getCompletedTasks() + 1);
workerRepository.save(worker);
```

Release 3 — Final Checklist

Release 3 complete when:
☐ Worker submits proof → image in Cloudinary, task status updated
☐ Citizen can verify or reject proof
☐ Auto-verification scheduler works
☐ Worker can retry up to 3 times on rejection
☐ Admin final approval → task COMPLETED
☐ Payment released → wallet balance updated, transaction recorded
☐ Citizen can rate worker (only post-completion)
☐ Worker profile shows updated average rating
☐ Test coverage > 80%

Release 4 — AI Intelligence & Event-Driven Architecture

Goal: Replace synchronous service-to-service calls with Kafka events. Add an AI service (FastAPI) that validates reports, classifies categories, detects duplicates, and scores priority. Add real-time WebSocket notifications.

New Infrastructure

Component	What to set up
Kafka	Broker + Zookeeper via Docker Compose. Topics: reports, bids, tasks, payments, notifications
Redis	Single node via Docker. Used for caching in R5 — just deploy now
FastAPI service	Separate Python service. Communicates via Kafka topics
WebSocket	Spring WebSocket + STOMP. Gateway at /ws

Event Format (all events)

DomainEvent (standard envelope)		
Field	Type	Notes
eventId	UUID	Unique per event
eventType	String	e.g. ReportCreated, BidPlaced
payload	JSON	Event-specific data
sourceService	String	e.g. report-service
timestamp	Instant	

Event Catalogue

Event	Topic	Published by
ReportCreated	reports	report-service
ReportSupported	reports	report-service
BidPlaced	bids	bid-service
TaskAssigned	tasks	task-service
ProofSubmitted	tasks	task-service
TaskCompleted	tasks	task-service
PaymentReleased	payments	payment-service
AIProcessingRequested	ai-requests	ai-gateway
AIProcessingResult	ai-results	ai-service (FastAPI)

Phase 1 — Event Bus Infrastructure

Kafka Setup (docker-compose addition)

```
zookeeper:
  image: confluentinc/cp-zookeeper:7.4.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

kafka:
  image: confluentinc/cp-kafka:7.4.0
  depends_on: [zookeeper]
  ports: ["9092:9092"]
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
```

Spring Kafka Config

```
# application.yml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
        org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: snapfix-backend
      auto-offset-reset: earliest
      value-deserializer:
        org.springframework.kafka.support.serializer.JsonDeserializer
```

Event Publisher Pattern

```
@Service
public class EventPublisher {
    @Autowired private KafkaTemplate<String, DomainEvent> kafka;

    public void publish(String topic, String eventType, Object payload) {
        DomainEvent event = new DomainEvent(UUID.randomUUID(), eventType,
            objectMapper.valueToTree(payload), "report-service", Instant.now());
        kafka.send(topic, event);
    }
}
```

Phase 1 — Definition of Done

- | |
|--|
| ☐ Kafka broker starts in Docker Compose |
| ☐ Spring Boot can publish to a topic |

- ☐ A test consumer receives the event
- ☐ Dead-letter queue configured for failed messages

Phase 2 — Event-Driven Architecture (Refactor)

Replace all direct service calls with event publishing. This is a refactor — existing APIs stay the same, internal communication changes.

Refactor Map

Before (direct call)	After (event)
reportService → notificationService.notify()	Publish ReportCreated → notification-service consumes
bidService → notificationService.notify()	Publish BidPlaced → notification-service consumes
taskService → notificationService.notify()	Publish TaskAssigned → notification-service consumes
paymentService → walletService.credit()	Publish PaymentReleased → wallet-service consumes

⚠ Do NOT refactor in one shot. Convert one flow at a time and run integration tests after each one.

Phase 2 — Definition of Done

☐ All notification triggers go through Kafka (no direct calls)
☐ Payment flow goes through event (wallet updated by consumer)
☐ Load test: 1,000 events/sec handled without loss

Phase 3 — AI Intelligence Service (FastAPI)

AI Stack

Component	Details
Framework	FastAPI (Python)
Image model	CLIP (openai/clip-vit-base-patch32 from HuggingFace)
Text model	SentenceTransformer (all-MiniLM-L6-v2)
Communication	Consumes from <code>ai-requests</code> Kafka topic, publishes to <code>ai-results</code>

AI Pipeline

19. Consume `AIProcessingRequested` event (contains image URL, description, location)
20. Download image from Cloudinary URL

21. Run CLIP — check image quality and civic relevance (if score < threshold → flag as invalid)
22. Run category classifier — predict: POTHOLE / STREETLIGHT / GARBAGE / WATER_LEAK / ROAD_DAMAGE
23. Run SentenceTransformer on description — store embedding for duplicate detection
24. Publish `AIProcessingResult` event with `{ reportId, isValid, predictedCategory, embedding }`

AI Gateway (Spring Boot side)

```
// In ReportService.createReport()
// After saving report to DB:
eventPublisher.publish("ai-requests", "AIProcessingRequested", Map.of(
    "reportId", report.getId(),
    "imageUrl", report.getImageUrl(),
    "description", report.getDescription(),
    "location", report.getLocation()
));

// Separate consumer listens on "ai-results" topic
// Updates report.category and flags invalid reports
```

Phase 3 — Definition of Done

☐ FastAPI service starts and connects to Kafka
☐ Report created → AI event triggered → category updated in DB
☐ Invalid image (e.g. selfie) → report flagged
☐ AI processing completes in < 3 seconds

Phase 4 — Duplicate Detection System

Detection Layers (all three must pass to flag as duplicate)

Layer	Implementation
Geo proximity	PostGIS <code>ST_DWithin</code> — radius 10 metres, same category
Text similarity	SentenceTransformer embeddings, cosine similarity > 0.85 threshold
Image similarity	CLIP embeddings, cosine similarity > 0.80 threshold

Embedding Storage

- Store report text embedding as `float[]` column (use `pgvector` extension for efficient similarity search)
- Add `CREATE EXTENSION IF NOT EXISTS vector;` migration
- Column type: `vector(384)` for MiniLM embeddings

Duplicate Handling

```
// In AI result consumer — after receiving AIProcessingResult
if (isDuplicate(reportId)) {
    Report original = findOriginal(reportId);
    reportService.addSupport(original.getId(), report.getCitizenId());
    reportService.delete(reportId); // or mark as DUPLICATE
    notifyCitizen(report.getCitizenId(), original.getId());
}
```

Phase 4 — Definition of Done

☐ pgvector extension installed and embeddings stored
☐ Duplicate submitted within 10m of existing → auto-support added
☐ Original report support count increments

Phase 5 — Priority Scoring Engine

Priority Score Formula

```
// PriorityService.calculateScore(report)
int supports      = report.getSupportCount();
int severity      = categoryToSeverity(report.getCategory()); // 1-5
long daysOpen     = ChronoUnit.DAYS.between(report.getCreatedAt(), Instant.now());
int ageFactor     = (int) Math.min(daysOpen * 2, 20); // cap at 20

int score = (supports * 2) + (severity * 5) + ageFactor;

// Map to priority level
// < 10 → LOW
// 10-24 → MEDIUM
// 25-39 → HIGH
// 40+ → CRITICAL
```

When Priority is Recalculated

- New support added to report
- Scheduled daily job recalculates age factor for all open reports

Phase 5 — Definition of Done

☐ priority field on Report entity (LOW/MEDIUM/HIGH/CRITICAL)
☐ Score recalculated when support added
☐ Daily scheduled job recalculates for all open reports
☐ Admin dashboard returns reports sorted by priority

Phase 6 — Real-Time Notification System

WebSocket Setup (Spring)

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue");
        config.setApplicationDestinationPrefixes("/app");
    }
    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").withSockJS();
    }
}
```

Push Notification Pattern

```
// In notification consumer (listens to all event topics)
@KafkaListener(topics = {"reports", "tasks", "payments"})
public void handleEvent(DomainEvent event) {
    List<String> recipients = resolveRecipients(event);
    for (String userId : recipients) {
        simpMessagingTemplate.convertAndSendToUser(
            userId, "/queue/notifications",
            buildNotificationPayload(event)
        );
        saveNotificationToDb(userId, event); // for history
    }
}
```

API Endpoints

Method	Endpoint	Description
GET	/notifications	Get notification history. Supports <code>?unread=true&page=0&size=20</code> .
PATCH	/notifications/{id}/read	Mark as read.
PATCH	/notifications/read-all	Mark all as read.

Release 4 — Final Checklist

Release 4 complete when:
☐ Kafka broker running, all events published and consumed
☐ Direct service-to-service notification calls replaced with events
☐ FastAPI AI service validates and classifies reports

☐ Duplicate detection auto-converts to support
☐ Priority score calculated and updated on reports
☐ WebSocket pushes real-time notifications to connected clients
☐ Event latency < 100ms (end-to-end)
☐ AI processing < 3 seconds
☐ Test coverage > 85%

Release 5 — Production Hardening & Platform Scalability

Goal: Make SnapFix deployable to production. API gateway, security hardening, observability, CI/CD, Kubernetes, and load testing.

Phase 1 — API Gateway Layer

All client requests go through one entry point. Spring Cloud Gateway handles routing, JWT validation, and rate limiting.

Route Configuration

```
# application.yml — Spring Cloud Gateway
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          predicates: [Path=/auth/**]
        - id: report-service
          uri: lb://report-service
          predicates: [Path=/reports/**]
          filters: [JwtAuthFilter]
        - id: worker-service
          uri: lb://worker-service
          predicates: [Path=/workers/**]
          filters: [JwtAuthFilter]
```

JWT Validation Filter

```
// GatewayFilter that validates JWT before forwarding
public class JwtAuthFilter implements GatewayFilter {
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        String token = extractToken(exchange.getRequest());
        if (!jwtUtil.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
        return chain.filter(exchange);
    }
}
```

Phase 1 — Definition of Done

- | |
|--|
| ☐ All backend services accessible only through gateway |
| ☐ Requests without valid JWT rejected at gateway (not at service level) |
| ☐ 1,000 concurrent requests handled without timeout |

Phase 2 — Security Hardening

Security Checklist

Implement all of these:
☐ Hibernate Validator annotations on all request DTOs (@NotNull, @Size, @Email)
☐ Global @ControllerAdvice returns 400 with field-level error messages on validation failure
☐ Add security headers via Spring Security: Content-Security-Policy, X-Frame-Options: DENY, X-Content-Type-Options: nosniff
☐ CORS policy: whitelist only frontend domain, not *
☐ JWT access token TTL: 15 min. Refresh: 7 days. No infinite tokens.
☐ Password: min 8 chars, 1 uppercase, 1 number, 1 special char (enforce via regex in DTO)
☐ Account lockout: 5 failed logins → lock for 15 minutes (track in Redis)
☐ OWASP ZAP scan — resolve all HIGH severity findings before release

Phase 3 — Observability System

Stack Setup

```
# docker-compose additions
prometheus:
  image: prom/prometheus
  volumes: ["/etc/prometheus/prometheus.yml"]
  ports: ["9090:9090"]

grafana:
  image: grafana/grafana
  ports: ["3000:3000"]
  depends_on: [prometheus]

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
  ports: ["9200:9200"]

kibana:
  image: docker.elastic.co/kibana/kibana:8.11.0
  ports: ["5601:5601"]
```

Metrics to Track (Grafana Dashboards)

Dashboard	Key Metrics
API Performance	p50/p95/p99 latency, error rate, req/sec per endpoint

Report Activity	Reports created/hour, categories breakdown, geo heatmap data
Task Lifecycle	Tasks in each state, avg time per state, retry rate
Worker Activity	Active workers, bids placed/hour, avg bid amount
Infrastructure	JVM heap, DB connection pool, Kafka consumer lag

Spring Boot Actuator + Prometheus

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
metrics:
  export:
    prometheus:
      enabled: true
```

Phase 3 — Definition of Done

☐ Prometheus scrapes metrics from all services
☐ Grafana shows API latency dashboard
☐ Kibana shows structured JSON logs searchable by requestId
☐ Alert configured: error rate > 5% triggers notification

Phase 4 — Rate Limiting & Caching

Redis Rate Limiting (at Gateway)

```
# Spring Cloud Gateway rate limiter filter
filters:
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 100      # tokens added per second
      redis-rate-limiter.burstCapacity: 200     # max burst
      key-resolver: "#{@userKeyResolver}"      # per user (JWT sub)
```

Redis Caching Strategy

Cache Key	TTL
<code>reports:nearby:{lat}:{lng}:{radius}</code>	60 seconds
<code>workers:{id}:profile</code>	5 minutes

reports:{id}:priority	2 minutes
notifications:{userId}:unread-count	30 seconds

```
// Spring Cache annotation (backed by Redis)
@Cacheable(value = "workerProfile", key = "#workerId")
public WorkerProfileDto getWorkerProfile(UUID workerId) { ... }

@CacheEvict(value = "workerProfile", key = "#workerId")
public void updateWorkerProfile(UUID workerId, ...) { ... }
```

Phase 4 — Definition of Done

☐ Rate limiter blocks requests > 100/min per user (returns 429)
☐ Nearby reports query hits Redis on second call within 60s
☐ Cache evicted when report/worker profile updated

Phase 5 — CI/CD Automation

GitHub Actions Pipeline

```
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgis/postgis:15-3.3
        env: { POSTGRES_PASSWORD: test }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: 21 }
      - run: ./mvnw test
      - run: docker build -t snapfix-backend .
      - run: docker push ghcr.io/${{ github.repository }}/snapfix-backend:${{
github.sha }}
```

Pipeline Rules

- `main` branch: any push triggers full pipeline
- PRs: must pass tests before merge is allowed (branch protection rule)
- Docker image tagged with Git SHA — never overwrite `latest` in CI
- Failed tests → pipeline stops, no image built

Phase 5 — Definition of Done

☐ Every push to main triggers CI pipeline
☐ Test failure blocks docker build
☐ Docker image pushed to GHCR on success

Phase 6 — Container Orchestration (Kubernetes)

Deployment Components

Component	Purpose
Deployment	Manages pod replicas for each service
Service (ClusterIP)	Internal service-to-service DNS routing
Ingress	External traffic → routes to gateway service
HorizontalPodAutoscaler	Scale pods when CPU > 70%
ConfigMap / Secret	Externalise env vars and credentials

Sample Deployment (backend)

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: snapfix-backend }
spec:
  replicas: 2
  selector: { matchLabels: { app: snapfix-backend } }
  template:
    spec:
      containers:
        - name: snapfix-backend
          image: ghcr.io/yourorg/snapfix-backend:latest
          resources:
            requests: { memory: 512Mi, cpu: 250m }
            limits: { memory: 1Gi, cpu: 500m }
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  maxReplicas: 8
  metrics: [{ type: Resource, resource: { name: cpu, target: { averageUtilization:
70 } } } ]
```

Phase 6 — Definition of Done

☐ All services deployed as Kubernetes pods

- ☐ HPA scales backend pods under load
- ☐ Rolling update: zero downtime deployment

Phase 7 — Load Testing

Use **k6** to validate performance under production-like load.

Test Scenarios

Scenario	Load	Pass Criteria
Concurrent browsing	1,000 users — GET /reports/nearby	p95 < 300ms, 0% errors
Report submission burst	500 POSTs in 60 seconds	p95 < 500ms, < 1% errors
Task completion flow	200 concurrent full flows	All complete, no data corruption
Stress test	Ramp to 2,000 users	Find breaking point, document it

k6 Script Skeleton

```
import http from "k6/http";
import { check } from "k6";

export const options = {
  stages: [
    { duration: "1m", target: 100 }, // ramp up
    { duration: "3m", target: 1000 }, // sustained load
    { duration: "1m", target: 0 }, // ramp down
  ],
  thresholds: {
    "http_req_duration": ["p(95)<300"],
    "http_req_failed": ["rate<0.01"],
  },
};

export default function () {
  const res =
    http.get("http://localhost:8080/reports/nearby?lat=18.6&lng=73.8&radius=5000");
  check(res, { "status 200": (r) => r.status === 200 });
}
```

Phase 7 — Definition of Done

- ☐ 1,000 concurrent users: p95 < 300ms on GET /reports/nearby
- ☐ System stable for 5-minute sustained load test
- ☐ Breaking point documented (for capacity planning)

- ☐ Slow queries identified and optimised (check EXPLAIN ANALYZE)

Phase 8 — City Analytics Dashboard

Backend — Analytics API

Method	Endpoint	Description
GET	/admin/analytics/reports	Reports per day/category/area. Params: <code>from</code> , <code>to</code> , <code>groupBy</code> .
GET	/admin/analytics/workers	Worker performance: completion rate, avg rating, tasks done.
GET	/admin/analytics/tasks	Avg resolution time, state distribution, retry rate.
GET	/admin/analytics/heatmap	Returns <code>[{ lat, lng, count }]</code> for map overlay.
GET	/admin/analytics/budget	Total payments per period and projected cost.

Frontend Stack

Library	Used for
React	Dashboard SPA
Chart.js / Recharts	Bar, line, pie charts
Mapbox GL JS	Issue heatmap and geo visualisation
React Query	Data fetching + caching for API calls

Release 5 — Final Checklist

Release 5 (Production Ready) complete when:
☐ All traffic routes through API gateway
☐ JWT rejected at gateway level
☐ Rate limiting active — 429 returned on abuse
☐ Redis caching measurably reduces DB load on hot queries
☐ OWASP ZAP: no HIGH/CRITICAL findings
☐ Grafana shows API latency dashboard
☐ Kibana shows structured logs searchable by requestId
☐ GitHub Actions CI runs on every push — failures block deploy
☐ All services running in Kubernetes with HPA
☐ k6 load test: p95 < 300ms at 1,000 concurrent users
☐ City analytics dashboard accessible to admin

☐ Test coverage > 90%

☐ 99.9% uptime simulation passed

SnapFix — All 5 releases complete.

The platform is production-grade, AI-powered, event-driven, and scalable.